```
expected = 0

   transitions = mdp.transitions(state, action)

   for successor, probability in transitions:

       reward = mdp.reward(state, action, successor)

       future_value = values[successor ]

       expected = (reward + (mdp.discount * future_value))*probability



   return expected
```

Revisa la implementación de action_value. Dime si es correcta

11:38 AM
Worked for 8s

No es correcta: en [planning.py (line 33)](/Users/laurafonseca/Uniandes/2026-2/ia/Taller3--IA-1/algorithms/planning.py:33) sobrescribes `expected` en cada iteración, por lo que devuelves únicamente la contribución del último sucesor.

Debes **acumular** las contribuciones usando `+=`:

```
None
expected += probability * (reward + mdp.discount * future_value)
```

Así calculas la suma esperada:

$$ Q(s,a)=\sum_{s'}P(s' \mid s,a)\,[R(s,a,s')+\gamma V(s')] $$

El resto de la función está bien, suponiendo que `values` contiene todos los estados y que los terminales tienen valor cero, como indica el enunciado.

```
expected = 0
   transitions = mdp.transitions(state, action)
   for successor, probability in transitions:
       reward = mdp.reward(state, action, successor)
       future_value = values[successor ]
```

```
      expected += (reward + (mdp.discount * future_value))*probability

  return expected
```

Quedo bien asi?

11:40 AM
Worked for 4s

Sí, ahora quedó correcta. Con `expected += ...` acumulas la contribución de **todos los sucesores**, ponderada por su probabilidad, tal como exige la ecuación de Bellman.

Solo un detalle de estilo: puedes cambiar `values[successor ]` por `values[successor]`; no afecta el resultado.

```
for _ in range(max_iterations):
    new_values = values.copy()
    delta = 0
    for state in mdp.states:
        actions = mdp.actions(state)
        if not actions:
            new_value = 0
        else:
            best_value = 0
            for action in actions:
                q = action_value(mdp, state, action, new_values)
                if q > best_value:
                    best_value = q
            new_value = best_value
        delta = max(delta, abs(new_value - values[state]))
        new_values[state] = new_value
    values = new_values
    converged = delta < tolerance
```

Revisa la implementacion de value_iteration. Dime si es correcta

11:57 AM
Worked for 13s

No del todo. Encontré **dos errores** en [value_iteration (line 41)](/Users/laurafonseca/Uniandes/2026-2/ia/Taller3--IA-1/algorithms/planning.py:41):

1. **Usa valores del barrido actual.** Al llamar `action_value(..., new_values)`, algunos estados ya están actualizados. Esto incumple el requisito de barridos sincrónicos y hace que el resultado dependa del orden de los estados. Debes pasar `values`:

None

2.
```
q = action_value(mdp, state, action, values)
```
3. **Descarta valores negativos.** Inicializar `best_value = 0` hace que, si todas las acciones tienen valor negativo, el estado quede incorrectamente en cero. Debes usar:

None

4.
```
best_value = -math.inf
```

Comprobé ambos con ejemplos pequeños: un estado recibió `0.9` cuando debía seguir en `0` durante el primer barrido; otro quedó en `0` y reportó convergencia cuando su valor correcto era `-1`.

La inicialización, los terminales en cero, el cálculo de `delta` y la condición de parada están bien. No modifiqué el archivo.

12:17 AM
Work locallyLocal
Ask for approval
Select effortGPT-6.1 SolLightNoneMinimalLightMediumHighExtra HighMaxUltraPersistent

```
  for _ in range(max_iterations):
    new_values = values.copy()
    delta = 0
    for state in mdp.states:
        actions = mdp.actions(state)
        if not actions:
            new_value = 0
        else:
            best_value = -math.inf
            for action in actions:
                q = action_value(mdp, state, action, values)
                if q > best_value:
                    best_value = q
            new_value = best_value
        delta = max(delta, abs(new_value - values[state]))
        new_values[state] = new_value
    values = new_values
    converged = delta < tolerance
```

Asi esta bien?

11:19 AM
Worked for 5s

Sí, **ahora `value_iteration` está correcta** para el modelo del proyecto.

Corregiste ambos errores: usas `values` para que los barridos sean sincrónicos e inicializas `best_value = -math.inf` para aceptar valores negativos. Los terminales, `delta` y la condición de parada también están bien.